

Exercise 2: Implementing the Factory Method Pattern

Scenario: Develop a document management system that creates different document types (Word, PDF, Excel) using the Factory Method Pattern.

1. Create a New Java Project

Create a Java project named `FactoryMethodPatternExample`.

2. Define Document Interface

```
public interface Document { void open(); }
```

3. Create Concrete Document Classes

```
public class WordDocument implements Document { public void open() {  
    System.out.println("Opening Word Document"); } } public class PdfDocument implements  
Document { public void open() { System.out.println("Opening PDF Document"); } } public class  
ExcelDocument implements Document { public void open() { System.out.println("Opening Excel  
Document"); } }
```

4. Implement the Factory Method

Abstract Factory Class:

```
public abstract class DocumentFactory { public abstract Document createDocument(); }
```

Concrete Factory Classes:

```
public class WordDocumentFactory extends DocumentFactory { public Document  
createDocument() { return new WordDocument(); } } public class PdfDocumentFactory extends  
DocumentFactory { public Document createDocument() { return new PdfDocument(); } } public class  
ExcelDocumentFactory extends DocumentFactory { public Document createDocument() { return  
new ExcelDocument(); } }
```

5. Test the Factory Method Implementation

```
public class FactoryMethodTest { public static void main(String[] args) { DocumentFactory  
wordFactory = new WordDocumentFactory(); Document wordDoc =  
wordFactory.createDocument(); wordDoc.open(); DocumentFactory pdfFactory = new  
PdfDocumentFactory(); Document pdfDoc = pdfFactory.createDocument(); pdfDoc.open();  
DocumentFactory excelFactory = new ExcelDocumentFactory(); Document excelDoc =  
excelFactory.createDocument(); excelDoc.open(); } }
```

Expected Output

Opening Word Document Opening PDF Document Opening Excel Document

Conclusion

The Factory Method Pattern creates objects without specifying their exact classes. Each factory is responsible for creating a specific document type, making the system flexible and easy to extend.